Steering the Linux Scheduler with Koopman Burst Prediction under EEVDF

Saarthak Manocha, Prathima Tirumalareddy and A Sirisha
Chaitanya Bharathi Institute of Technology, Hyderabad, India
saarthakmanocha@gmail.com, prathimareddy.t@gmail.com, asirishanaidu@gmail.com

Abstract

Shortest Job First (SJF) scheduling needs the length of each task's next Central Processing Unit (CPU) burst before the burst runs. This paper predicts bursts with a Koopman operator, fitted by Dynamic Mode Decomposition (DMD) on log-scale delay coordinates, and feeds the predictions to the unmodified Linux Earliest Eligible Virtual Deadline First (EEVDF) scheduler as per-task slice requests. Every measurement is validated against the kernel's own accounting, on a virtual machine (CPU time within 0.93%, 105 processes) and on a 28-CPU bare-metal machine (within 0.44% for every process with at least 0.5 s of CPU time, 903 processes, no lost events). On real programs (Redis, SQLite, Node.js, ffmpeg) the predictor beats the textbook exponential average in 32 of 36 threads, with 29% lower error. The predictor is shown to be numerically a log-scale autoregressive model, and neither nonlinear observables, an Autoregressive Integrated Moving Average (ARIMA) model nor a neural network beats that model. In closed loop on bare metal with 1, 8 and 16 CPUs, an Extended Berkeley Packet Filter (eBPF) observer that costs at most 1.4% of one core feeds the predictor, and predicted slices cut the response time of short requests by 57–59% with no loss of throughput; the Koopman predictor beats the exponential average in 48 of 54 paired runs. Slices need a margin above the predicted burst: a slice exactly equal to the true burst gives only 8–9%, while 1.5 times the burst gives 72%. With real servers sharing a core with CPU-bound tasks on a second bare-metal machine, Redis requests became 43% faster, SQLite point queries 77% and Node.js light requests 34%, although Redis's 99th-percentile latency rose by 21%. A complete sched_ext scheduler built on the same predictor cut the response time of short requests by 44% and of long requests by 41% against stock Linux. The capture kit, data, controller, sched_ext scheduler and a scheduler visualizer are released.

Index Terms— Koopman operator, Dynamic Mode Decomposition, CPU scheduling, Earliest Eligible Virtual Deadline First, Shortest Job First, Extended Berkeley Packet Filter, kernel tracing, conformal prediction

I. Introduction

Shortest Job First (SJF) scheduling minimizes the mean waiting time on one processor, and its preemptive form, Shortest Remaining Time First (SRTF), does the same when jobs arrive over time [1]. Both need something no operating system has: the length of each task's next CPU burst before it runs. Standard operating-systems textbooks therefore teach an estimate of it, the *exponential average* of the previous bursts [1], [2], which Section II-A explains; this paper calls it the *textbook estimator*. Production schedulers have avoided prediction altogether. The Linux kernel's default scheduler from kernel version 2.6.23 (2007) until kernel version 6.6 was the Completely Fair Scheduler (CFS), which always runs the task that has received the least CPU time so far [3]. In kernel version 6.6 (October 2023) its task-selection logic was replaced by Earliest Eligible Virtual Deadline First (EEVDF) [4], [5], which serves tasks in order of virtual deadlines that depend on how much CPU time they have received and on a time slice, not on how long their next burst will be.

Since kernel version 6.12 (November 2024) the slice is no longer fixed. A task may request its own slice through the `sched_setattr` system call, and a shorter slice gives it an earlier virtual deadline and the right to preempt tasks with longer slices [6], [7]. This turns burst prediction from a textbook exercise into a lever on a production scheduler. If a task's next burst is known to be short, its slice can be set short, and EEVDF will serve it sooner; that is SJF expressed inside the stock kernel, with no patch and no new scheduling class. Whether this works depends on two questions that, to the authors' knowledge, have not been answered with real kernel data: how well can bursts be predicted, and does a better prediction make the real scheduler faster?

This paper studies both with Koopman operator methods [8], [9], [10]. A Koopman operator represents a nonlinear system by a linear operator acting on functions of its state; Dynamic Mode Decomposition (DMD) and Extended Dynamic Mode Decomposition (EDMD) estimate it from data [11], [12], and its eigenvalues describe how patterns in the data decay or oscillate. Burst sequences are short, noisy and nonstationary, which suits a model that is linear in a well-chosen space, can be updated online in nanoseconds, and comes with a spectrum that describes each task. Koopman models on delay coordinates are, however, closely related to autoregressive models [13], [14], so a third question is what the Koopman view adds beyond them.

Approach. A measurement pipeline was built first, and every reconstructed quantity was validated against counters that the kernel keeps independently; this exposed two hazards that would otherwise have corrupted the data. Burst predictors were then compared on synthetic workloads, a bare-metal trace and four real programs, against ten baselines. Finally the loop was closed: a controller observes each task's bursts, predicts the next one and sets its EEVDF slice, first by polling the `/proc` file system and then through an Extended Berkeley Packet Filter (eBPF) program attached to the scheduler's tracepoint. The controller was evaluated in 504 paired runs: 264 on a two-core virtual machine, 180 on a 28-CPU bare-metal machine with 1, 8 and 16 CPUs in use, and 60 with real servers on a second, 8-CPU bare-metal machine. A sched_ext scheduler driven by the same predictor was then evaluated in 36 further runs.

Findings. (i) On real programs, a log-scale Koopman predictor beats the textbook estimator in 32 of 36 threads, with 29% lower error. (ii) It is numerically a log-scale autoregressive model, and neither nonlinear Koopman dictionaries, an Autoregressive Integrated Moving Average (ARIMA) model nor a neural network beats an autoregressive model with enough delays: what predicts CPU bursts is the log scale and the delay depth. (iii) Fed to EEVDF as slice requests, predictions cut the response time of short requests by 57–59% on bare metal with 1, 8 and 16 CPUs at negligible cost, and the latency of real servers' short requests by 34–77%. (iv) How a prediction becomes a slice matters more than which predictor is used: a slice exactly equal to the true burst gives only 8–9%, because the burst overruns its slice, while 1.5 times the burst gives 72%. A short model explains this, and also predicts where a more accurate predictor helps, which the experiments confirm. (v) The same predictor can drive a complete scheduler: a sched_ext policy that orders tasks by virtual time plus predicted burst beat stock EEVDF by 44% on short requests and 41% on long ones, while the same policy without predictions was far worse than stock Linux.

Contributions.

1. A validated measurement pipeline for scheduler dynamics: trace capture without perturbation, ground truth from the kernel's `schedstat` counters, self-tests, and two documented measurement hazards (Section V).

2. A burst-prediction study on synthetic workloads, a bare-metal trace and four real programs, against autoregressive, ARIMA, neural, gradient-boosting and nonlinear-Koopman baselines, with prediction intervals and per-prediction cost (Section VI).

3. A closed-loop controller that steers the unmodified EEVDF scheduler through slice requests, with an eBPF burst observer, a model of the slice margin, and an evaluation in 504 paired runs on synthetic and real workloads, on a virtual machine and two bare-metal machines (Sections IV and VII).

4. A complete sched_ext scheduler driven by the predictor, evaluated on bare metal (Section VII-G).
5. A multi-core analysis, a machine-level Koopman change detector and an interactive trace visualizer (Sections VIII–X).

Section XI summarizes applications of the same estimator above the operating system, and all code and data are released.

II. Background

A. The textbook burst estimator

Operating-systems textbooks introduce burst prediction together with SJF scheduling [1], [2]. The length of the next CPU burst is not known in advance, but bursts of the same process tend to be similar in length, so the next one is predicted from the lengths of the previous ones by the *exponential average*

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \quad 0 \le \alpha \le 1,$$

where t_n is the length of the n-th (most recent) burst, τ_n the previous prediction and τ_{n+1} the prediction for the next burst; the first prediction τ_0 is a constant, such as a system-wide average. The parameter α sets the weight of recent against past history: with $\alpha = 0$ the latest burst is ignored, with $\alpha = 1$ only the latest burst counts, and the common textbook choice $\alpha = 1/2$ weights the latest burst and the past history equally. Expanding the recursion,

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\alpha\, t_{n-1} + \ldots + (1-\alpha)^j\,\alpha\, t_{n-j} + \ldots + (1-\alpha)^{n+1}\,\tau_0,$$

so each older burst counts geometrically less than the one after it [1]. Tanenbaum and Bos describe the same technique as *aging*, with $\alpha = 1/2$, which can be computed with one addition and one shift [2]. In statistics and forecasting the same recursion is known as exponential smoothing [15] or the Exponentially Weighted Moving Average (EWMA) [16].

Throughout this paper, the *textbook estimator* or *textbook EWMA* means this exponential average with $\alpha = 0.5$, the form taught in these textbooks; a variant with α tuned on training data is reported as well. It is the natural baseline: it is what an operating-systems course proposes for SJF, and it is cheap enough to run inside a scheduler. Its weakness, examined in Section VI, is that it averages on a linear scale, so a single long burst pulls the next few predictions far upwards.

B. EEVDF and slice requests

EEVDF [4] gives each runnable task i a virtual runtime v_i, which advances by the CPU time it receives divided by its weight. The run queue's average $V = \sum_i w_i v_i / \sum_i w_i$ defines each task's *lag* $V - v_i$, the service it is owed. A task is *eligible* when its lag is non-negative, and its virtual deadline is

$$d_i = v_i + \frac{s_i}{w_i},$$

where s_i is its slice. The scheduler runs the eligible task with the earliest deadline. A task that sleeps keeps its lag, so on waking it is placed at $v_i = V - \mathrm{lag}_i$, and a task that has slept while others ran is usually eligible at once. When a running task has used its slice, its deadline is renewed, $d_i \leftarrow v_i + s_i / w_i$, and the scheduler picks again.

Two features matter here. With *run to parity*, the task picked keeps the CPU until its slice is used, even if another task becomes more deserving, which limits context switches. With slice requests (kernel version 6.12) [6], [7], any task may set s_i between 0.1 ms and 100 ms through the `sched_runtime` field of `sched_setattr`; a woken task whose slice is shorter than the running task's, and whose deadline is earlier, may preempt it at once. The default slice on the test machine is 1.4 ms. A shorter slice is therefore a request to be served sooner, paid for by more frequent preemption if the burst does not fit.

C. Koopman operators and Dynamic Mode Decomposition

For a discrete-time system $x_{t+1} = F(x_t)$, the Koopman operator K acts on observables g of the state by composition, $(Kg)(x) = g(F(x))$ [8]. It is linear even when F is not, at the price of acting on a space of functions. Its eigenfunctions evolve by multiplication with their eigenvalue, so the eigenvalues describe growth, decay and oscillation of patterns in the data [9]. EDMD [12] approximates K on a finite dictionary ψ of observables by the matrix

$$K = \arg\min_A \| \Psi_Y - \Psi_X A \|_F = \Psi_X^{+} \Psi_Y ,$$

where the rows of Ψ_X and Ψ_Y are $\psi(x_t)$ and $\psi(x_{t+1})$. With $\psi(x) = x$, EDMD is DMD [11], [17]; as the data grow it converges to the projection of K onto the span of the dictionary [18]. When only a scalar is measured, delay coordinates $(y_t, y_{t-1}, \dots)$ reconstruct the state (Takens' theorem) [19], and DMD on them (Hankel DMD) approximates the Koopman spectrum for ergodic systems [13]. Linear models on delay coordinates include autoregressive models and Hankel DMD as special cases [14], [20]. Measurement noise biases DMD eigenvalues towards zero [21]. Koopman models have been used for prediction and control in fluids, power systems and finance [22], [23], [24], and learned dictionaries extend them with neural networks [25].

D. Extended Berkeley Packet Filter and sched_ext

Extended Berkeley Packet Filter (eBPF) programs run verified code inside the kernel at tracepoints and other hooks, and communicate with user space through maps and ring buffers [26]. sched_ext (kernel version 6.12) lets an eBPF program implement a complete scheduling policy, with the kernel reverting to its built-in scheduler if the program fails [27].

III. Related Work

Burst prediction for scheduling. The exponential average of Section II-A is the standard textbook estimator [1], [2]. Negi and Kumar predicted CPU bursts of known programs from static and dynamic process attributes with decision trees and set Linux 2.4 time slices from the predictions, reducing turnaround time by 1.4–5.8% [28]. That work required a modified kernel and per-program training; the present work uses the stock scheduler's slice interface and predicts each task from its own recent bursts. Size-based scheduling has long been known to help workloads with heavy-tailed job sizes [29].

Learned and pluggable schedulers. Reinforcement learning has been used to schedule jobs in data-processing clusters [30]. On a single machine, sched_ext has made experimental policies easy to deploy [27]; a recent system uses a learned workload classifier to switch between whole sched_ext policies at run time [31]. The present work instead keeps the stock scheduler and changes only the slice it is asked to use. The Linux scheduler has a history of subtle performance bugs that only careful measurement revealed [32], which motivates the emphasis on validation here.

Koopman methods. Koopman operators have been applied to prediction and control of physical and financial systems [22], [23], [24], [10]. The relation between DMD on delay coordinates and autoregressive

models is known [13], [14], and classical time-series practice selects the autoregressive order by information criteria [33], [34]. The authors are not aware of Koopman methods applied to operating-system scheduling; this paper tests the method against these classical alternatives rather than assuming an advantage.

IV. Method

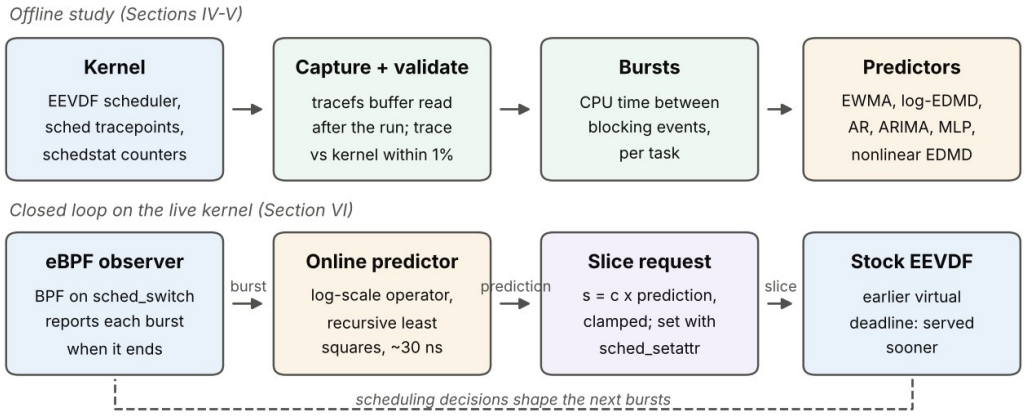


Fig. 1. Overview. Top: offline study of traced bursts and predictors. Bottom: the closed loop on the live kernel, in which an eBPF program reports each burst, a per-task operator predicts the next one, and the prediction becomes an EEVDF slice request.

Fig. 1 shows the two parts of the study. This section defines the predictor, the mapping from prediction to slice, and the model that explains why that mapping needs a margin.

A. Burst predictor

A *burst* is the CPU time a task uses between two moments at which it blocks or sleeps; preemptions do not end a burst. For each task, the state is its last p bursts on a log scale,

$$z_t = \left(\log b_t, \log b_{t-1}, \ldots, \log b_{t-p+1}\right),$$

standardized with training statistics, and a single operator is fitted by least squares,

$$K = \arg\min_A \sum_t \| z_{t+1} - z_t A \|^2 .$$

The predicted next burst is the first coordinate of $z_t K$, mapped back from the log scale. The delay depth is $p = 3$ unless stated otherwise. The log scale matters: burst lengths span several orders of magnitude, and on a linear scale a few long bursts dominate the least-squares fit and every forecast.

Because the dictionary is linear in delay coordinates, the forecast equals that of a log-scale Autoregressive (AR) model of order p, AR(p), fitted by least squares; the two differ only through centring and an intercept. Across the 96 tasks studied, the median of the largest difference between their log-scale forecasts is 10^{-3}. The predictor is called log-EDMD below. Two nonlinear dictionaries that an AR model cannot express are also tested: the lags plus their squares and pairwise products (*poly-EDMD*), and the lags plus 64 Random Fourier Features (RFF) $\sqrt{2/D}\cos\left(z_t W + b\right)$ with Gaussian W, an approximation of kernel EDMD (*RFF-EDMD*).

Online form. In the controller each task's operator is updated after every burst by recursive least squares with an intercept and forgetting factor $\lambda = 0.995$. With regressor $x_t = (1, \log b_t, \log b_{t-1}, \log b_{t-2})$ and target $y = \log b_{t+1}$,

$$g \quad ¿ \frac{P x_t}{\lambda + x_t^\top P x_t}, \theta \leftarrow \theta + g\left(y - \theta^\top x_t\right),$$
$$P \qquad \leftarrow \left(P - g x_t^\top P\right)/\lambda,$$

and the next prediction is $\exp\left(\theta^\top x_{t+1}\right)$. Until a task has ten bursts, the textbook estimator is used.

Spectrum. The eigenvalues of K summarize a task's burst dynamics: a dominant eigenvalue near one indicates a slowly decaying level, complex pairs indicate alternation, and eigenvalues near zero indicate bursts close to independent.

B. From prediction to slice

The slice requested for a task is

$$s = min\left(max\left(c\,\hat{b}, s_{min}\right), s_{\text{def}}\right),$$

where $\hat{b}$ is the predicted burst, c a margin, $s_{min} = 0.1$ ms the kernel minimum and s_{def} the default slice (1.4 ms here). The controller only ever shortens slices. Algorithm 1 gives the loop.

Algorithm 1. Slice controller (eBPF version)

1. **eBPF program, on** `sched_switch`(prev, next, prev_state):
2. if prev is tracked: acc[prev] += now − on[prev]
3. if prev_state ≠ RUNNING: emit (prev, acc[prev]); acc[prev] ← 0
4. if next is tracked: on[next] ← now
5. **User space, for each event** (task i, burst b):
6. update task i's operator with b (recursive least squares)
7. $\hat{b} \leftarrow$ predicted next burst of task i
8. $s \leftarrow min\left(max\left(c\,\hat{b}, s_{min}\right), s_{\text{def}}\right)$
9. `sched_setattr`(i): `sched_runtime` ← s

C. Why the slice needs a margin

Consider a request task that wakes while CPU-bound tasks with the default slice share its core. Its response time R, from wake-up to the end of its burst, has three parts. The *wait* $W(s)$ before it first runs grows with its slice s: its deadline is $v + s$, so a smaller s makes it more likely to preempt the running task at once and to be picked first. Its *CPU demand* b' is the burst b plus system overhead (wake-up path, system calls, accounting). And if $b' > s$, its deadline is renewed while it still runs and it loses its place to tasks whose deadlines are now earlier, adding a *penalty* Δ of the order of their slices. Thus

$$E[R \mid s] \approx W(s) + b' + \Delta \, Pr(b' > s).$$

With $s = c\,\hat{b}$ and a log-normal prediction error $\log(b/\hat{b}) \sim N(0, \sigma^2)$, and with small overhead,

$$Pr(b' > s) \approx 1 - \Phi\left(\frac{\log c}{\sigma}\right).$$

The first term pushes c down and the third pushes it up, so there is an interior optimum. Two predictions follow. First, even a perfect predictor ($\sigma = 0$) needs $c > 1$, because $b' > b$: a slice of exactly the burst should overrun often and give little or no gain. Second, at a fixed margin, a predictor with smaller σ overruns less; so differences between predictors should appear at small c, where overruns are frequent, and vanish at large c, where every slice saturates at the default. Section VII tests both.

D. Uncertainty

A scheduler can act on an interval instead of a point. Split conformal prediction [35], [36] is used for this: on a calibration set, the scores $|\log(b/\hat{b})|$ give a quantile q, and the interval $[\hat{b}\,e^{-q}, \hat{b}\,e^{q}]$ covers a new burst with probability at least $1 - a$ if bursts are exchangeable. Bursts are a time series, so the coverage is measured rather than assumed.

V. Measurement and Validation

TABLE I. Platforms and data

Platform	Kernel version	Capture	Runs
Lab, 28 CPUs, bare metal (Trace 3)	7.0	perf	1 (75 s)
Lab, Intel Core i7-14700, 28 CPUs, bare metal	7.0	tracefs, counters	42 + 24
same machine, closed loop	7.0	counters, eBPF	180
Lab, Intel Core i7-7700, 8 CPUs, bare metal	7.0	tracefs, eBPF, sched_ext	3 + 60 + 36
Intel Xeon Virtual Machine (VM), 2 virtual CPUs	6.18	tracefs	12 + 24 + 3
same VM, closed loop	6.18	counters, eBPF	264
ARM VM, 4 virtual CPUs (Apple host)	6.8	counters	9 + 24

Platforms (Table I). A 28-CPU bare-metal lab machine (kernel version 7.0) recorded Trace 3. A second lab machine, used for the bare-metal scaling, frequency, change-detection and closed-loop experiments, has an Intel Core i7-14700 with 8 performance cores of two hardware threads each and 12 efficiency cores (28 CPUs; Ubuntu 26.04, kernel version 7.0); CPUs 0–15 are the performance-core threads, as their peak clock of 5.0–5.45 GHz against 4.2 GHz for CPUs 16–27 shows, and the default EEVDF slice is 2.8 ms. A third lab machine, used for the real-program captures, the real-server loop and the sched_ext scheduler, has an Intel Core i7-7700 with 4 cores of two hardware threads each (8 CPUs; Ubuntu 24.04, kernel version 7.0); its tracing self-test also saw every switch (201 out, 201 in). The virtual platforms are an Intel Xeon Virtual Machine (VM) with two virtual CPUs under the Kernel-based Virtual Machine (KVM) hypervisor (kernel version 6.18, 250 Hz tick, no kernel preemption) and an ARM virtual machine with four virtual CPUs on an Apple host (kernel version 6.8). All run EEVDF.

Workloads. Synthetic workloads are small programs: a CPU-bound loop; an Input/Output (I/O)-bound loop that writes 10 kB and sleeps 50 ms; and a *phased* positive control that alternates every 2 s between bursts of 1 ms and 20 ms of CPU time, so its bursts have known structure. The lab trace (*Trace 3*) ran three CPU-bound and three I/O-bound jobs. VM captures ran these plus two phased jobs, pinned to 1 or 2 cores, overloaded, or with a scheduled load drop, each three times. Real programs and the closed-loop workloads are described in Sections VI and VII.

Reconstruction. From the `sched_switch`, `sched_wakeup` and `sched_migrate_task` tracepoints, every task's history is rebuilt: every interval on a CPU with its state at switch-out (preempted, sleeping or waiting on disk), every interval waiting in a run queue after a wake-up or a preemption, and every migration. Consecutive intervals that end in preemption are merged into one burst.

Ground truth. Each workload saves, just before it exits, the kernel's own `/proc/self/schedstat` (CPU time, run-queue wait and number of times scheduled) and its voluntary and involuntary context-switch counts. These are maintained by the scheduler independently of tracing. Over 105 processes in 12 VM captures (Fig. 2), reconstructed CPU time differs from the kernel's by 0.93% (median; maximum 3.45%), wait time by 0.22% (maximum 2.07%), and the number of times scheduled by a median of 4 out of thousands, with no lost events. The small excess of traced CPU time comes from the exit path after the counters are read and from interrupt time, which the kernel's per-task accounting excludes.

The 42 bare-metal captures of the second lab machine (903 processes; Section VIII) were validated in the same way. CPU time agrees with the kernel within 0.44% for every process that used at least 0.5 s of CPU (median 0.02%), the number of times scheduled agrees exactly at the median, and no events were lost. The trace exceeds the kernel by a nearly constant 4 ms per process (median), the exit path; for I/O jobs that use only about 40 ms of CPU in a run this is a large fraction, so their agreement is better judged in absolute terms (all within 20 ms). On cores that are often idle the trace counts up to about 12 μs more wait per wake-up than the kernel's run-queue delay; with every core busy the two agree within 1 μs per wake-up. The bare-metal scaling results below are therefore taken from the kernel's own counters.

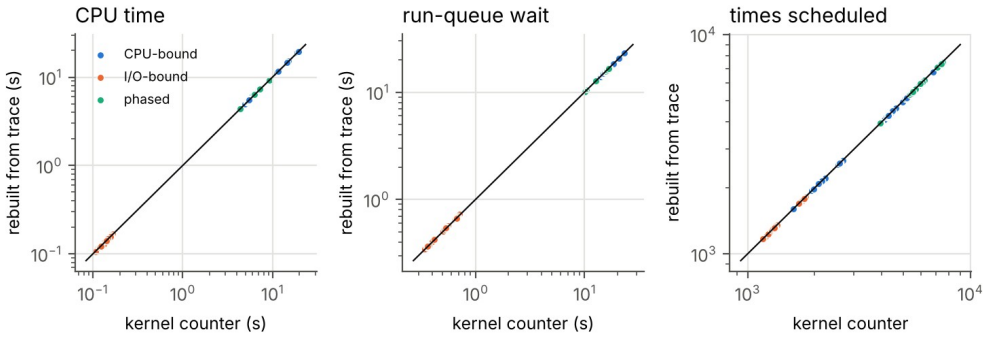


Fig. 2. Every reconstructed process against the kernel's own counters (105 processes, 12 captures).

Hazard 1: the observer effect. A first capture tool streamed events out of the kernel while recording. Each event woke the reader, which preempted the workload and produced further events: a feedback loop of about 50,000 context switches per second on any core with spare capacity. Traces grew from 0.5–0.8 MB to 15–23 MB and counts disagreed with the kernel by up to 9%. Recording into a large kernel buffer and reading it once after the run removed the loop. One contaminated capture is kept as evidence.

Hazard 2: missing idle exits on a VM. On the Intel VM's kernel, part of the `sched_switch` events for a CPU leaving idle are never emitted; tracefs and perf agree, and the ring buffer reports no loss. In seven self-tests a task that slept 201 times was seen switching back in only 1 to 176 times. None of the lab machines shows it: there were zero unmatched switch-outs in Trace 3, in all 42 captures of the second lab machine and in the 3 captures of the third, and their self-tests saw every switch back in (204 of 204 and 201 of 201). The

cause was not determined. The parser therefore counts unmatched switch-outs, the capture tool runs a self-test, and VM experiments keep every core busy, either with workload or with one lowest-priority (`SCHED_IDLE`) spinner per core, which receives only CPU time nothing else wants. With busy cores the counts match the kernel: in a 15 s real-program test without spinners there were 1,639 unmatched switch-outs, and with them 186 across three 60 s captures and 310,000 timeslices.

VI. Burst Prediction

A. Protocol

Every method predicts a burst from earlier bursts of the same task only; the last 20% of each task is the test set and everything is fitted on the rest. Methods: training mean and median; last value; the textbook estimator (EWMA with $\alpha = 0.5$, Section II-A) and EWMA with α tuned on training data; log-EDMD, poly-EDMD and RFF-EDMD; recursive (multi-step) EDMD; log-AR with order 1–10 chosen by the Akaike Information Criterion (AIC) [34] (log-AR(p*)); ARIMA(p, d, q) with $p \le 2$, $d \le 1$, $q \le 1$ chosen by AIC [33]; a neural network with two hidden layers of 32 units on six log lags; and gradient boosting on three raw lags. Errors are mean absolute errors; ratios to a reference are computed per task. Intervals are 95% moving-block bootstrap intervals with blocks of 10 [37]; across tasks, sign tests with Holm correction are used [38].

B. Synthetic workloads and the bare-metal trace

Uncontended bursts (Trace 3). On the idle lab machine (on average 2.3 of 28 cores busy), CPU-bound bursts ended only because kernel threads preempted them: all 769 preemptions came from `kworker` (55%) or `migration` threads (45%). I/O bursts roughly tripled, from 85–113 μs to 320–340 μs, on the *same* cores when the CPU-bound jobs exited (Fig. 3). Power management was suspected, and a later test on the second lab machine supports it. The same workload (three CPU-bound jobs for 40 s, I/O and phased jobs for 70 s, all cores) ran three times with the default CPU frequency governor and three times with the governor fixed at `performance`. With the default governor, the I/O bursts lengthened in 9 of 9 processes once the CPU-bound jobs exited, by a median factor of 1.65 (median burst 61 μs before, 101 μs after). With the `performance` governor the lengthening disappeared (59 μs before, 57 μs after; median per-process ratio 0.87). The effect is smaller than the threefold change in Trace 3, which was recorded on a different machine, but it has the same direction and depends on the power-management setting. This level shift dominates I/O forecasting: methods that follow recent bursts cut error by 52–83% against a constant, and the textbook estimator was best or tied. The I/O operators' dominant eigenvalues, 0.88–0.97, are the Koopman signature of the shift, and recursive EDMD forecasts decay to the mean, as eigenvalues inside the unit circle imply.

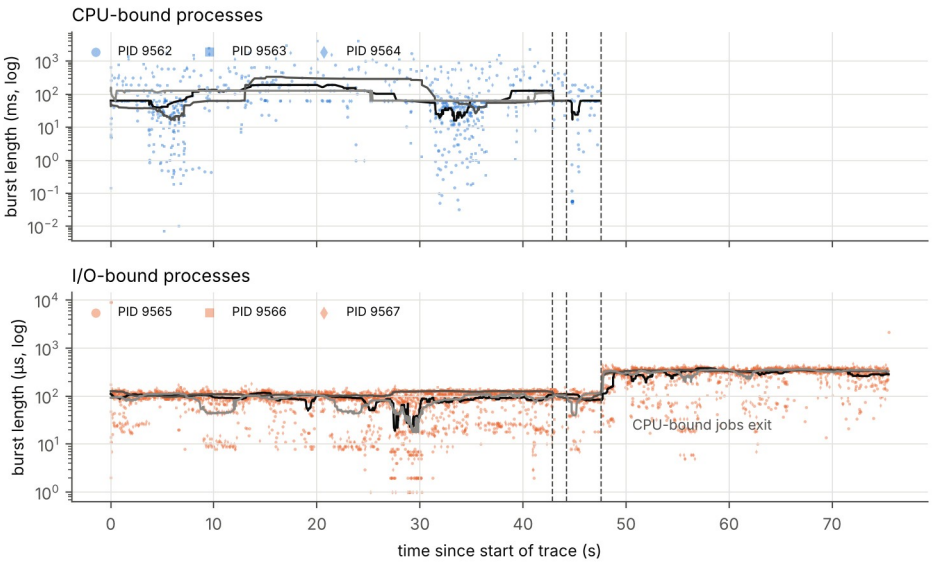


Fig. 3. Trace 3 (bare metal, 28 cores): every burst over time. I/O bursts lengthen about threefold on the same cores when the CPU-bound jobs exit (dashed lines).

TABLE II. Burst prediction, median error ratio to the textbook estimator (EWMA, $\alpha = 0.5$) and number of tasks where the method is better

Method	phased (24)	contended I/O (33)	real programs (36)
mean	14.6, 0	0.97, 22	0.86, 26
last value	0.62, 24	1.26, 0	1.21, 0
EWMA, tuned α	0.64, 24	1.08, 9	0.94, 29
EDMD (linear scale)	1.09, 0	0.93, 26	0.82, 29
poly-EDMD	1.05, 4	0.90, 32	0.81, 35
log-EDMD	1.23, 0	0.86, 33	0.71, 32
gradient boosting	1.24, 0	0.82, 33	0.79, 34

Bursts under contention (VM). Results over 57 processes are in Table II and Fig. 4. On the positive control (24 phased processes), every method that uses recent bursts cuts error by 90–96% against the mean, so the methods find structure when it exists; the best is tuned EWMA (better than $\alpha = 0.5$ in 24 of 24), and the EDMD variants are 5–23% *worse* than the textbook estimator. On I/O bursts under contention (33 processes) the order reverses: log-EDMD beats the textbook estimator in 33 of 33 processes (median error ratio 0.86), poly-EDMD in 32 and gradient boosting in 33; with Holm correction over ten comparisons, $p_{\text{Holm}} = 2 \times 10^{-9}$ for log-EDMD and gradient boosting and 6×10^{-8} for poly-EDMD. Contended I/O bursts are noisy, and the exponential average overreacts to single long bursts; the log-scale operator does not.

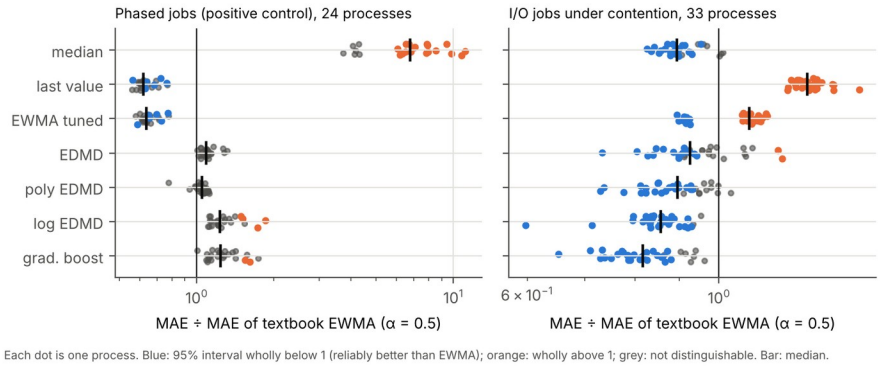


Fig. 4. Contended VM workloads: error relative to the textbook estimator, one dot per process. Blue/orange: 95% interval wholly below/above 1.

C. Real programs

To test beyond synthetic loops, the following programs were traced together on both VM cores in three captures of 60 s: a Redis 7.0 server driven by a paced client (mostly GET and SET, with occasional range and sort commands on a 30,000-element list); a Node.js 22 Hypertext Transfer Protocol (HTTP) server whose client alternates between phases of light and heavy JavaScript Object Notation (JSON) requests; a Python process running Structured Query Language (SQL) queries of mixed size (point lookups, range aggregates, full scans) against SQLite 3.45; real-time H.264 encoding of a synthetic video with ffmpeg (all encoder threads); and the two load generators. A parallel gcc build and xz compression ran as untracked background load. Over the 36 threads with at least 60 bursts, CPU time agrees with `schedstat` within 1.8% and wait time within 1.5% (medians); 28 threads are within 5%, and the others are ffmpeg helper threads that run about 12 μs per slice, where a few microseconds of tracepoint overhead is a large fraction.

Log-EDMD beats the textbook estimator in 32 of 36 threads, with median error ratio 0.71 ($p_{\mathrm{Holm}} = 10^{-5}$; Table II, Fig. 5). The gain is largest for Redis and SQLite (median ratios 0.55 and 0.56), whose request bursts are short with occasional long ones: the exponential average chases each long burst and then overestimates the short ones that follow, while the log-scale operator treats a long burst as a large but transient deviation. The same four programs were captured three times on the 8-CPU bare-metal machine. For the 24 threads that used at least 0.5 s of CPU, traced CPU time agreed with the kernel within 3.1% (median 0.33%), with no lost events; idle ffmpeg helper threads, with a few tens of microseconds of CPU each, are not comparable. Log-EDMD beat the textbook estimator in 24 of 30 threads (median error ratio 0.81, sign test $p = 0.0014$), again most for Redis and SQLite (0.55 and 0.56, as on the VM) and the load generators (0.71), and least for ffmpeg (0.94). The spectral picture repeated too: SQLite's eigenvalues were again real, and eigenvalues identified the program a window came from with 77% balanced accuracy, against 99% for simple burst statistics.

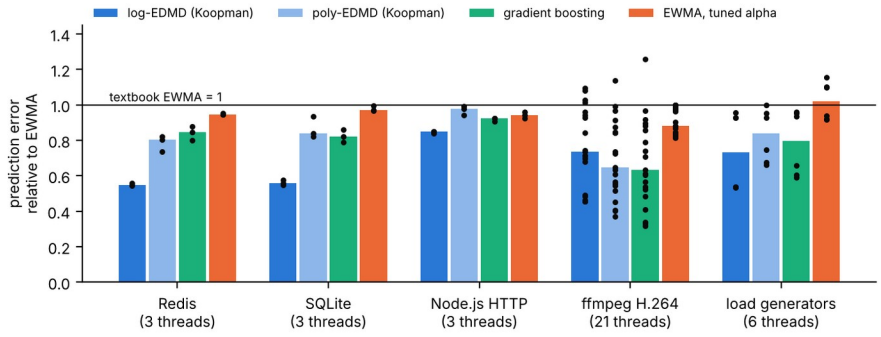


Fig. 5. Real programs: burst-prediction error relative to the textbook estimator (bars: median; dots: threads; three captures).

D. Is it more than an autoregressive model?

TABLE III. Median error ratio and number of tasks where the method is better, against two references (96 tasks, 36 of them real-program threads)

	vs textbook EWMA		vs log-AR(p*)	
Method	all 96	real 36	all 96	real 36
log-EDMD (3 delays)	0.88, 65	0.71, 32	1.02, 21	1.01, 8
log-AR(p*), AIC	0.79, 66	0.70, 32	–	–
poly-EDMD	0.86, 68	0.71, 35	1.01, 41	1.00, 20
RFF-EDMD (kernel)	0.85, 60	0.74, 28	1.00, 46	1.00, 20
ARIMA, AIC	0.85, 66	0.71, 33	1.02, 31	1.00, 15
neural network	0.84, 66	0.74, 32	1.02, 34	1.00, 18
gradient boosting	0.87, 68	0.79, 34	1.03, 33	1.04, 14

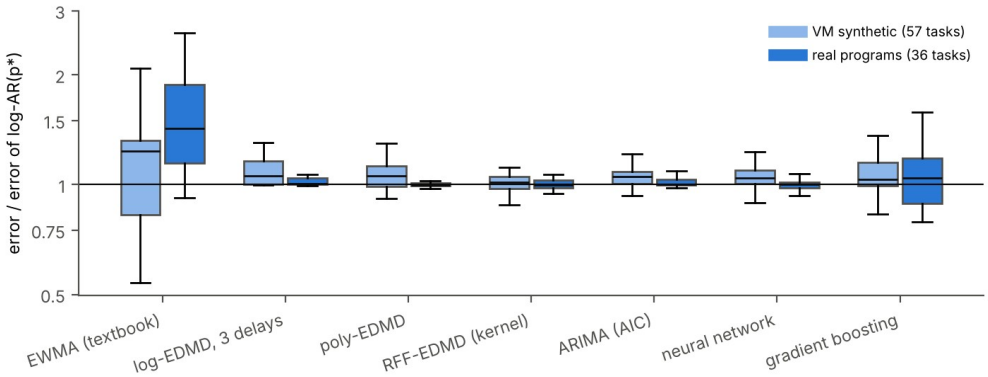


Fig. 6. Error of each model relative to a log-scale autoregressive model with AIC-chosen order (boxes: quartiles; whiskers: 1.5 interquartile ranges). Only the textbook estimator is clearly worse.

Table III and Fig. 6 compare all models over the 96 tasks with at least 60 bursts (57 synthetic, 3 from Trace 3, 36 real). Against the textbook estimator, every model that uses recent bursts wins, and all by similar amounts. Against the strongest classical model, log-AR with AIC-chosen order, none wins. The AIC most often chose order 10 (49 of 96 tasks), and log-AR(p*) is better than log-EDMD with three delays in 75 of 96 tasks, though only by 1.5% at the median; this is Hankel DMD with more delay coordinates. Nonlinear dictionaries do not help: poly-EDMD and RFF-EDMD tie with log-AR(p*) (Holm $p = 0.37$ and 0.76), and ARIMA, the neural network and gradient boosting are slightly worse. For CPU bursts, what predicts is the log scale and the delay depth, not nonlinearity. The Koopman formulation remains useful for the online update and for the spectrum.

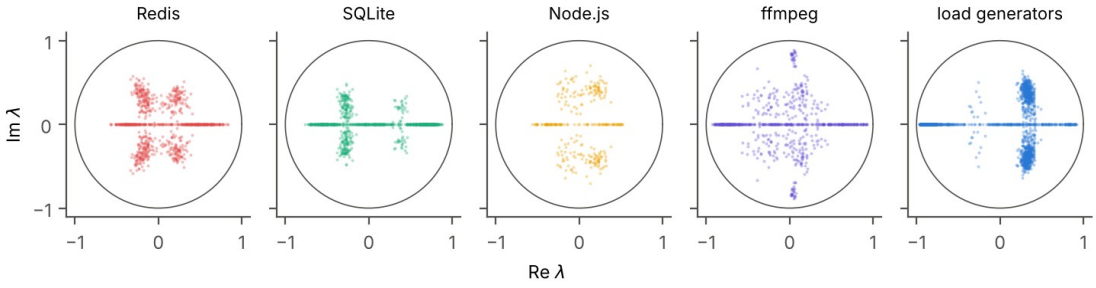


Fig. 7. Eigenvalues of the log-scale operator fitted on windows of 150 bursts, per real program (three captures).

What the spectrum shows. Fig. 7 plots the eigenvalues of operators fitted on windows of 150 bursts. Each program has a recognisable signature. SQLite's eigenvalues are mostly real (the median window's largest imaginary part is 0), spread along the real axis from about -0.7 to 0.85. The load generators have the slowest real mode (median $|\lambda_1| = 0.87$) and a cluster of complex pairs near $0.3 \pm 0.35i$, from their alternation between sending and waiting. Redis and Node.js decay fastest (median $|\lambda_1| \approx 0.45$), with complex pairs indicating alternating short and long bursts. The eigenvalues alone identify which of the five programs a window came from with 76% balanced accuracy in leave-one-capture-out cross-validation (chance 20%), although simple burst statistics (log mean, log spread, lag-one autocorrelation) do better (99.8%). The spectrum is therefore best read as a description of burst dynamics, not as a classifier.

E. Uncertainty and cost

TABLE IV. Split conformal intervals (target 90%) and cost per prediction

Tasks	Method	Coverage	Width e^{2q}
I/O (36)	log-EDMD	0.90	5.7
I/O (36)	textbook EWMA	0.91	8.3
phased (24)	log-EDMD	0.93	1.29
phased (24)	textbook EWMA	0.93	1.17
Cost: C, predict next burst		5.6 ns	
Cost: C, recursive-least-squares update		29 ns	
Cost: Python, predict		8.2 μs	
Cost: Python, refit on 300 bursts		41 μs	

For 60 processes from the validated captures (fit on 60%, calibrate on 20%, test on 20%), conformal intervals reached the target 90% coverage despite the time-series structure (Table IV). The log-EDMD interval was narrower than the textbook estimator's in 34 of 60 processes (median width ratio 0.84), clearly so for I/O tasks. In C, a three-lag operator predicts the next burst in 5.6 ns and updates itself in 29 ns, less than a context switch; cost is not an obstacle to running a Koopman predictor inside a scheduler.

VII. Closing the Loop: Steering EEVDF

The controller was first developed and evaluated on the two-core Intel VM (Sections VII-B and VII-C); the eBPF experiment was then repeated on the bare-metal lab machine with 1, 8 and 16 CPUs (Section VII-D). Real servers were steered on the VM (Section VII-E) and on a second bare-metal machine (Section VII-F), and Section VII-G replaces the scheduler itself with a sched_ext policy driven by the same predictor.

A. Set-up

Workload. Request tasks, written in C, wake on a fixed period of 15, 20 or 25 ms and do one CPU burst each, timed with the thread CPU clock. Burst lengths come from a two-regime log-autoregressive process (coefficient 0.6, noise 0.35): short bursts near 0.3 ms and long ones near 4 ms, switching regime with probability 0.05 per request. Each request logs its deadline, wake-up time and finish time; if it finishes after the next deadline, the next request starts at once. CPU hogs (CPU-bound background tasks) spin and count the CPU time they receive, which measures throughput. All tasks are pinned to the CPUs under test.

Design. Every repetition runs every policy with the same random seeds, in shuffled order, so comparisons are paired, and Wilcoxon signed-rank tests [39] are used. The tasks' own CPU accounting agrees with `schedstat` within about 1% (median per policy). The main metric is the mean response time of short

requests (under 1 ms of CPU), from deadline to finish; long requests, all requests, wake-up latency, context switches and hog throughput are also reported.

Policies. Stock Linux (default slice for every task); a 0.1 ms slice for every task (*fixed*); a 0.1 ms slice for request tasks only, set by hand (*hand-labelled*); predictors setting slices from the textbook estimator (EWMA) or log-EDMD, with margin c or with a conformal margin (the prediction times the 90th percentile of the task's recent error ratios); and oracles, in which each task sets its own slice to c times its true next burst.

B. Polling agent (VM)

TABLE V. Polling agent: mean response of short requests (ms) and change against stock Linux; change in CPU hogs' throughput

Policy	1 CPU (8 runs)	2 CPUs (6 runs)	hogs, 1 CPU
stock Linux	2.14	2.22	–
fixed 0.1 ms	2.15 (0%)	2.25 (+2%)	−0.6%
hand-labelled 0.1 ms	2.07 (−3%)	2.17 (−2%)	−0.7%
textbook EWMA	1.52 (−29%)	1.58 (−29%)	−3.5%
log-EDMD	1.48 (−31%)	1.59 (−28%)	−4.6%
textbook EWMA, conformal	1.39 (−35%)	1.49 (−33%)	−6.3%
log-EDMD, conformal	1.71 (−20%)	1.76 (−21%)	−7.0%
oracle, $c=1$	2.20 (+3%)	2.29 (+3%)	−0.7%
oracle, $c=2$	1.16 (−46%)	1.19 (−46%)	−1.1%

A first agent polled `/proc/<pid>/schedstat` and `status` every 10 ms and estimated each burst from CPU time and voluntary context switches. On one CPU (three hogs, four request tasks, eight repetitions of 20 s) and two CPUs (six hogs, eight request tasks, six repetitions; 635,531 requests in all), both predictors cut short-request response in every paired repetition (Table V, Fig. 8), by 28–31% (Wilcoxon $p=0.008$ and 0.03, the smallest values attainable with 8 and 6 pairs). Koopman was 2.6% better than the textbook estimator on one CPU (7 of 8 pairs, $p=0.016$) and level on two. Hand-labelled and uniformly short slices cut wake-up latency (median 0.12 ms against 1.82 ms) but chop every burst into 0.1 ms pieces, with 7–8 times more context switches, and do not shorten responses. Long requests took 4–7% longer, the usual cost of SJF. CPU hogs lost 3–7% of their CPU time, which matches the agent's own CPU time almost exactly (0.57 s lost against 0.60 s used by the log-EDMD agent in 20 s): the cost of a Python program polling `/proc`, not of the scheduling decision. The oracles already show the effect explained in Section IV-C: an exact slice gives nothing, twice the burst gives 46%.

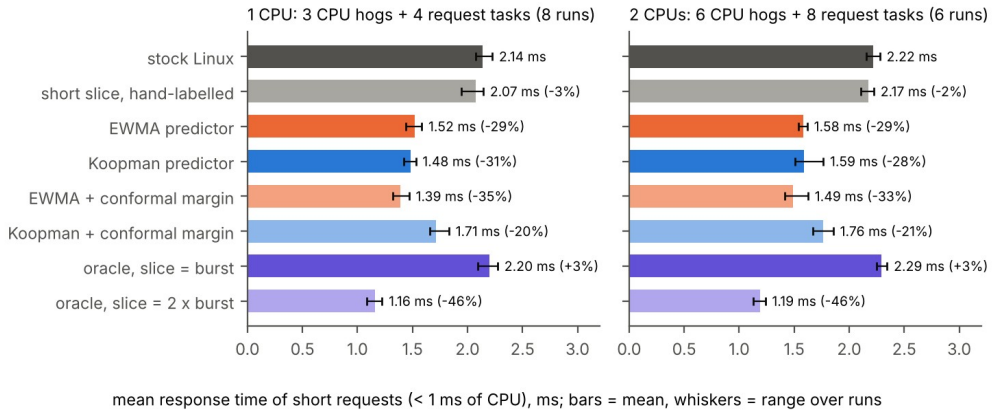


Fig. 8. Polling agent on the stock EEVDF scheduler (two-core VM): mean response time of short requests per slice policy (bars: mean over paired repetitions; whiskers: range).

C. eBPF agent and the slice margin (VM)

TABLE VI. eBPF agent, one CPU, six paired runs per policy: short-request response (ms, change against stock Linux), median wake-up latency (ms) and context switches per second

Policy	*C*	short-request response	wake-up p50	switches/s
stock Linux	–	2.27	1.88	1296
oracle	1	2.30 (+1%)	0.30	1735
	1.5	0.89 (−61%)	0.93	1552
	2	1.27 (−44%)	1.31	1509
	3	1.74 (−23%)	1.54	1374
textbook EWMA	1	1.52 (−33%)	0.38	2020
	1.5	1.34 (−41%)	0.83	1869
	2	1.47 (−35%)	1.19	1780
	3	1.90 (−16%)	1.50	1711
log-EDMD	1	1.45 (−36%)	0.29	2024
	1.5	**1.25 (−45%)**	0.78	1862
	2	1.42 (−37%)	1.22	1782
	3	1.92 (−15%)	1.53	1681

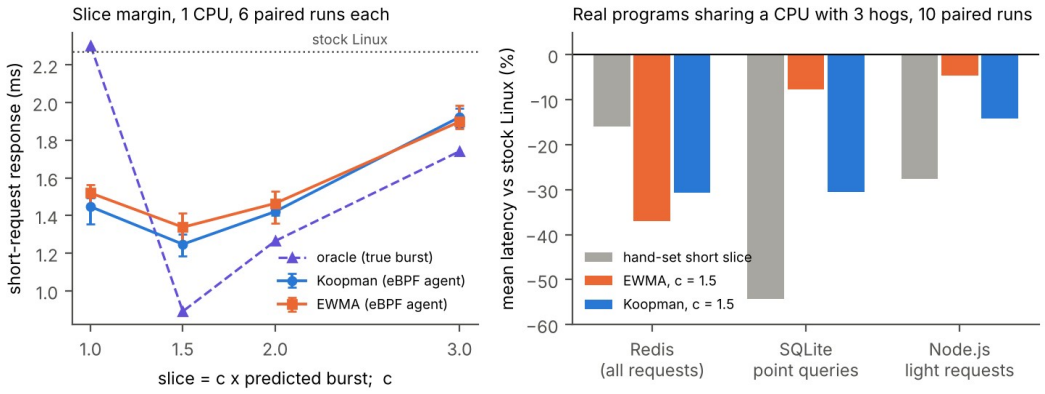


Fig. 9. Left: short-request response against the slice margin c (eBPF agent, one VM CPU with three hogs; bars: range over six paired runs); the oracle knows each burst exactly. Right: change in mean latency of real programs sharing one CPU with three hogs (ten paired runs), against stock EEVDF. Both panels: two-core VM.

An eBPF program on the `sched_switch` tracepoint reports every burst of a tracked task the moment it ends, through a ring buffer, and a C agent updates the task's operator and sets its slice (Algorithm 1). The agent uses 0.33% of one core, against about 3% for the polling agent, and the CPU hogs' throughput stayed within ±1 % of stock Linux under every predictor.

Table VI and Fig. 9 (left) sweep the margin, and both predictions of Section IV-C hold. The oracle with $c=1$ is no better than stock Linux, although its wake-up latency is the lowest of all (0.30 ms against 1.88 ms): bursts overrun their slices and lose their place. With $c=1.5$ the oracle gives −61 %, and larger margins lose ground as wake-up latency rises (−44 % at 2, −23 % at 3). The predictors follow the same curve, with the best margin also at 1.5: log-EDMD −45 % (1.25 against 2.27 ms) and the textbook estimator −41 %. Mean response over *all* requests also improved, by 5–6%, and long requests were not slower (21.9 against 22.4 ms). Log-EDMD beat the textbook estimator in 6 of 6 paired runs at $c=1$ and at $c=1.5$ ($p=0.031$ each), 5 of 6 at $c=2$ and 2 of 6 at $c=3$; over all margins, in 19 of 24 ($p=0.0035$). As the model predicts, the advantage of the more accurate predictor appears where slices are tight and vanishes where they saturate. Its burst predictions were 9% more accurate (mean absolute error 0.77 against 0.84 ms) and their log-errors less spread (0.49 against 0.58).

D. Bare metal: 1, 8 and 16 CPUs

TABLE VII. eBPF agent on bare metal (Intel Core i7-14700, kernel 7.0): mean response of short requests (ms) and change against stock Linux, six paired runs per policy; three CPU-bound hogs and four request tasks per CPU

Policy	*C*	1 CPU	8 CPUs	16 CPUs
stock Linux	–	2.26	2.30	2.33
oracle	1	2.05 (−9%)	2.10 (−8%)	2.15 (−8%)
	1.5	0.63 (−72%)	0.65 (−72%)	0.65 (−72%)
	2	0.72 (−68%)	0.70 (−69%)	0.71 (−70%)
textbook EWMA	1	1.28 (−43%)	1.35 (−41%)	1.36 (−42%)
	1.5	1.09 (−52%)	1.14 (−50%)	1.14 (−51%)
	2	1.04 (−54%)	1.08 (−53%)	1.09 (−53%)
log-EDMD	1	1.25 (−45%)	1.29 (−44%)	1.31 (−44%)
	1.5	**0.97 (−57%)**	**0.98 (−57%)**	**0.98 (−58%)**
	2	0.99 (−56%)	0.97 (−58%)	0.95 (−59%)

Policy	c	1 CPU	8 CPUs	16 CPUs
log-EDMD better than EWMA		13 of 18	18 of 18	17 of 18
agent CPU (% of one core)		0.09	0.6	1.2–1.4
hog throughput vs stock		within ±0.8% for every policy		

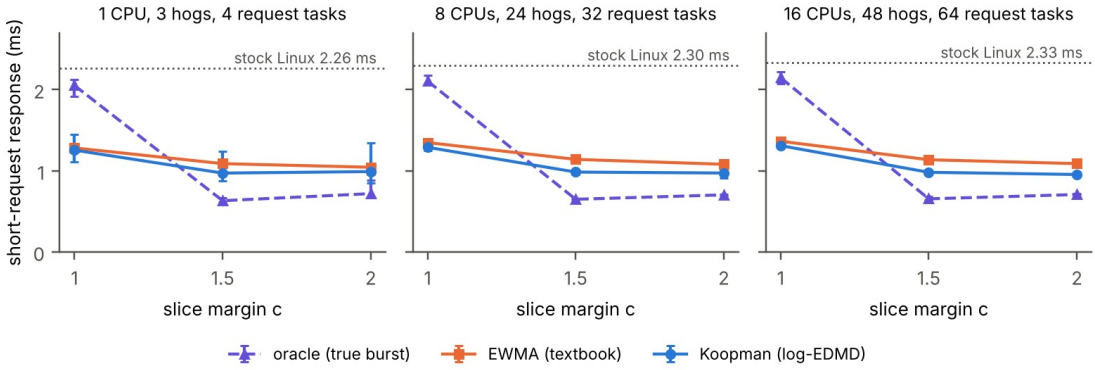


Fig. 10. Bare metal: short-request response against the slice margin c with 1, 8 and 16 CPUs (bars: range over six paired runs). The oracle knows each burst exactly; dotted line: stock Linux.

The eBPF experiment was repeated on the second lab machine with 1, 8 and 16 CPUs (CPU 0, CPUs 0–7 and CPUs 0–15, all performance-core threads), three CPU-bound hogs and four request tasks per CPU, the same request workload, all ten policies and six paired repetitions of 20 s: 180 runs. The request tasks' own CPU accounting agrees with `schedstat` within 0.3% (median per policy).

The effect is larger than on the VM and almost identical across core counts (Table VII, Fig. 10). Predicted slices cut the mean response of short requests by 57–58% with log-EDMD at $c=1.5$ and by 56–59% at $c=2$; the textbook estimator gave 50–54%. Every predictor policy beat stock Linux in 6 of 6 pairs on every core count ($p=0.031$, the smallest value attainable with six pairs). Log-EDMD beat the textbook estimator at every margin and core count: by 11–14% at $c=1.5$, by 10–12% at $c=2$ on 8 and 16 CPUs and by 2–4% at $c=1$, and in 48 of 54 paired runs overall (Wilcoxon $p=9\times10^{-8}$), including 18 of 18 on 8 CPUs and 17 of 18 on 16 CPUs. Its burst predictions were 10% more accurate (mean absolute error 0.73 against 0.81 ms).

Both predictions of Section IV-C hold again. A slice exactly equal to the true burst gains only 8–9%, while 1.5 times the burst gives 72%; the predictors do best at $c=1.5$–2. The default slice on this machine is 2.8 ms rather than 1.4 ms, so slices reach the default only at larger margins, and the advantage of the more accurate predictor persists at $c=2$ (6 of 6 pairs on 8 and on 16 CPUs) instead of fading as it did on the VM.

Costs stayed small. Hog throughput was within ±0.8% of stock Linux under every policy, and the agent used 0.09%, 0.6% and 1.2–1.4% of one core while tracking 4, 32 and 64 tasks. Long requests took 0.2–2.9% longer with log-EDMD at $c=1.5$, and the mean over all requests fell by 3.5–5.7%. The 99th percentile of short-request response fell from 8.9–9.3 ms to 6.0 ms with log-EDMD at $c=1.5$ (textbook estimator: 7.5–7.7 ms), and median wake-up latency fell from 2.6–2.8 ms to 0.36–0.47 ms.

E. Real programs (VM)

TABLE VIII. Real programs sharing one CPU with three hogs: change in mean latency against stock Linux (ten paired runs, margin $c=1.5$)

Requests	stock (ms)	hand-set	textbook EWMA	log-EDMD
Redis GET	0.63	−27%	−51%	−44%
Redis SET	0.63	−23%	−48%	−40%
Redis LRANGE	5.84	+10%	+2%	+5%

Requests	stock (ms)	hand-set	textbook EWMA	log-EDMD
Redis p99	7.59	+9%	0%	+1%
SQLite point	1.68	−54%	−8%	−31%
SQLite range	5.28	+4%	−2%	+4%
SQLite scan	343	0%	−1%	+2%
SQLite p99	427	−4%	−3%	−2%
Node.js light	4.25	−28%	−5%	−14%
Node.js heavy	180	−3%	−2%	0%
Node.js p99	334	0%	0%	+4%

A Redis server, a Node.js server and a SQLite worker shared one CPU with three hogs, while their clients ran on the other CPU and timed every request end to end (Redis, Node.js) or from its scheduled start (SQLite). The agent controlled the three server threads. Table VIII and Fig. 9 (right) give the results over ten paired repetitions.

Predicted slices made short requests faster in all three programs. Redis requests overall became 31% (log-EDMD) and 37% (textbook estimator) faster, SQLite point queries 31% and 8%, and Node.js light requests 14% ($p = 0.01$) and 5% (not significant); the log-EDMD changes for Redis and SQLite held in 10 of 10 pairs ($p = 0.002$). Log-EDMD was 25% better than the textbook estimator on SQLite point queries (10 of 10 pairs) and 10% worse on Redis (1 of 10). Heavy requests, the overall mean of Node.js and SQLite, and the 99th percentiles of all three programs changed by less than 4% under either predictor (not significant), and hogs lost 1% of their CPU time. A hand-set minimum slice for the three servers did best for SQLite point queries (-54%) and Node.js light requests (-28%) but worst for Redis (-16%, with its 99th percentile 9% higher), and it requires knowing in advance which tasks are latency-sensitive; the predictors needed no such knowledge.

F. Real servers on bare metal

TABLE IX. Real servers on bare metal (Intel Core i7-7700, kernel 7.0), sharing one CPU with three hogs: change against stock Linux (ten paired runs, margin $c = 1.5$)

Requests	stock (ms)	hand-set	textbook EWMA	log-EDMD
Redis GET	0.61	−61%	−66%	−63%
Redis SET	0.60	−58%	−64%	−63%
Redis LRANGE	6.06	+24%	+4%	+22%
Redis, all	0.77	−39%	−48%	−43%
Redis p99	7.52	+27%	−1%	+21%
SQLite point	1.89	−82%	−23%	−77%
SQLite range	4.18	+39%	+9%	+32%
SQLite scan	298	+2%	+3%	+3%
SQLite p99	321	+1%	+2%	+1%
Node.js light	4.20	−35%	−34%	−34%
Node.js heavy	147	−1%	+0.5%	−1%
Node.js p99	254	0%	+5%	−1%

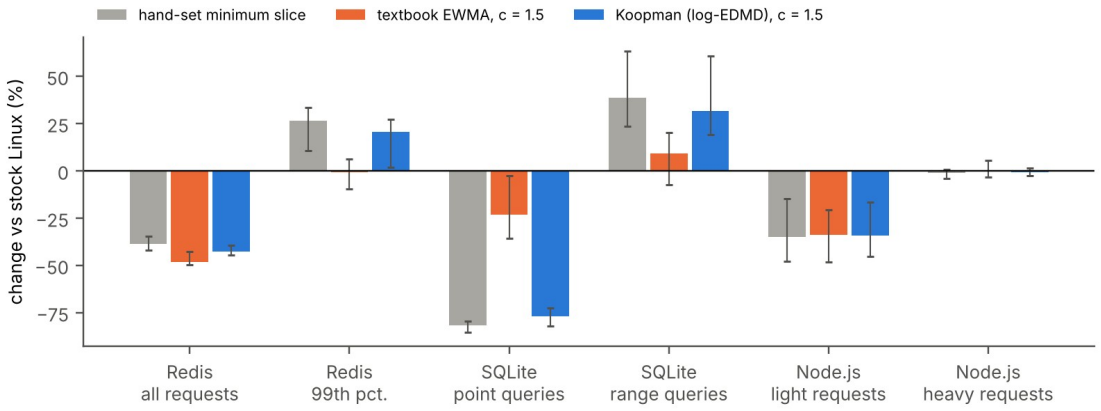


Fig. 11. Real servers on bare metal sharing one CPU with three hogs: change against stock EEVDF (bars: mean of ten paired runs; whiskers: range of the per-run changes).

The real-server experiment was repeated on the 8-CPU bare-metal machine, with the servers on CPU 0 and their clients on CPU 1 as before, ten paired repetitions of six policies (Table IX, Fig. 11). The effect was larger than on the VM. With log-EDMD at $c = 1.5$, Redis requests overall became 43% faster (GET and SET 63%), SQLite point queries 77% and Node.js light requests 34%; the textbook estimator gave 48%, 23% and 34%. Every one of these changes held in 10 of 10 pairs ($p = 0.002$). Log-EDMD's point queries were 70% faster than the textbook estimator's (10 of 10 pairs), while on Redis overall it was 11% slower (better in 1 of 10). Heavy requests and SQLite scans changed by at most 3%, hogs lost 1.1% of their CPU time, and the agent used 0.4% of one core.

The cost appeared in medium-sized requests. With log-EDMD, Redis LRANGE requests took 22% longer, SQLite range queries 32% longer and Redis's 99th-percentile latency rose by 21% (10 of 10 pairs), while the textbook estimator raised them by at most 9%. The log-scale operator predicts short bursts for these servers more confidently, so an occasional medium request overruns its slice and loses its place, the penalty Δ of Section IV-C. The hand-set minimum slice showed the same pattern more strongly (+24%, +39% and +27%). A margin that grows with the spread of a task's recent bursts would address this; it was not tested.

G. A Koopman scheduler in the kernel (sched_ext)

TABLE X. sched_ext scheduler on bare metal (Intel Core i7-7700, 8 CPUs, 24 hogs, 32 request tasks; six paired runs): mean response (ms) and change against stock EEVDF; hog throughput change

Policy	short requests	long requests	all requests	short, p99	hogs
stock EEVDF	2.51	20.6	11.5	10.0	–
sched_ext, no prediction	35.8 (+1327%)	42.8 (+107%)	39.2 (+242%)	49.9 (+397%)	+34%
sched_ext, EWMA, margin 1.5	1.56 (−38%)	12.8 (−38%)	7.16 (−38%)	11.5 (+14%)	−0.3%
sched_ext, log-EDMD, margin 1	1.35 (−46%)	15.0 (−27%)	8.11 (−29%)	9.66 (−4%)	+1.2%
sched_ext, log-EDMD, margin 1.5	**1.41 (−44%)**	12.3 (−41%)	6.82 (−41%)	9.11 (−9%)	−0.5%
sched_ext, log-EDMD, margin 2	1.55 (−38%)	12.0 (−42%)	6.73 (−41%)	10.1 (+1%)	−1.9%

Slice requests change only one input of EEVDF. The release also contains `scx_koopman`, a complete sched_ext scheduler in which the prediction decides the order as well: an eBPF program keeps one global queue ordered by virtual time plus predicted next burst, measures every burst, and gives each task a slice of the margin times its prediction, while a user-space agent updates each task's log-scale operator by recursive least squares, as in Algorithm 1. Tasks without a prediction get the sched_ext default slice of 20 ms. It was run on the 8-CPU bare-metal machine with 24 hogs and 32 request tasks, six paired repetitions of six policies (Table X). The kernel loaded and unloaded it cleanly in all 30 runs that used it.

Without predictions the same scheduler was far worse than stock Linux: every task, hogs included, received 20 ms slices, hogs gained 34% more CPU time, and short requests took 14 times longer. With predictions it beat stock EEVDF on every request type. Log-EDMD at margin 1.5 cut the mean response of short requests by 44% and of long requests by 41% (6 of 6 pairs each), lowered the 99th percentile of short requests by 9%, and left hog throughput within 0.5%. It was 9% faster than the textbook estimator on short requests (6 of 6 pairs, $p = 0.031$), whose 99th percentile was 14% worse than stock. The price is context switches, 5.6 times as many as stock Linux. The margin behaves as in Section IV-C: a margin of 1 was best for short requests but cost long ones, and margins of 1.5–2 balanced the two.

VIII. Multi-Core Scheduling

Measured latency. On the lab machine, processes waited a median 16 μs for a CPU; no queue formed, so the order in which a scheduler picks tasks could not matter. On one VM core the same kind of workload waited a median 0.07 ms (I/O) and 2.6 ms (phased) after waking, with 95th percentiles of 2.6 and 7.9 ms. On the ARM VM, using only the kernel's per-process counters, CPU-bound jobs spent 76% of their time waiting on one core, 50% on two and 1% on four (Fig. 12); the Intel VM traces give the same values on one and two cores.

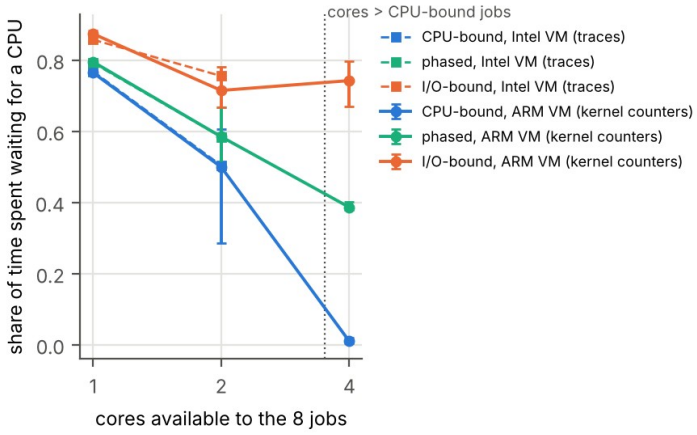


Fig. 12. Share of time spent waiting for a CPU, same 8 jobs on 1, 2 and 4 cores, two virtual machines, three repetitions.

Bare-metal scaling. On the second lab machine the same eight jobs (three CPU-bound, three I/O-bound, two phased) ran on 1, 2, 4, 8, 16 and all 28 CPUs, three times each, and the kernel's own counters give the waiting (Fig. 13, left). CPU-bound jobs spent 76.5% of their time waiting on one CPU, 50.2% on two, 0.05% on four and none from eight up, the same values as on both virtual machines. A second series added one CPU-bound job per CPU, so that every core always had CPU-bound work (Fig. 13, right). The phased interactive jobs then waited 1.75–1.77 ms per run at 4, 8 and 16 CPUs and 1.56 ms at 28, against under 1 μs per run when 8 or more cores were free. Adding cores does not reduce the waiting of interactive tasks when every core already holds CPU-bound work: what matters is the order in which each core picks its tasks, and

that is what a slice request changes. Under overload (two CPU-bound jobs per CPU) the interactive jobs waited 2.6 ms (phased) and 0.18 ms (I/O) per run.

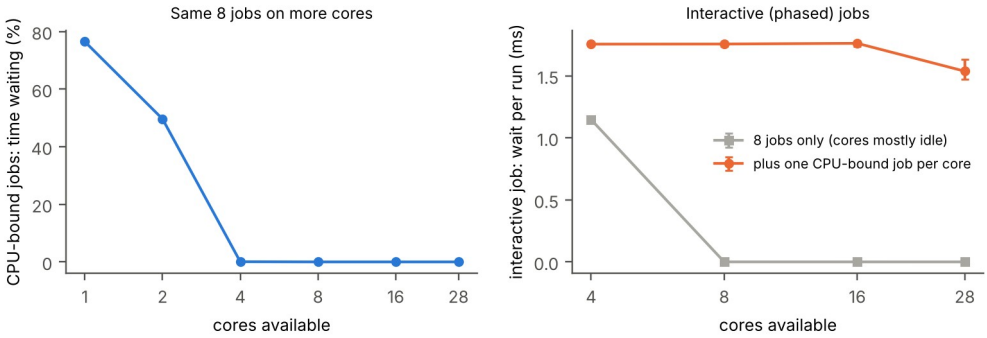


Fig. 13. Bare metal, 28 CPUs, kernel counters, three repetitions (bars: range). Left: CPU-bound jobs' share of time spent waiting, same 8 jobs on 1–28 CPUs. Right: waiting per run of the phased interactive jobs, with the cores otherwise free (grey) and with one CPU-bound job per core (orange).

Trace-driven simulation. Each traced task is replayed as a closed loop of its real CPU bursts and blocking times on m identical cores with one shared queue, under First-Come, First-Served (FCFS), Round Robin (3 ms time quantum), non-preemptive SJF and preemptive SRTF, each with a predictor (running mean, textbook EWMA, online log-EDMD or exact lengths). A job that has run past its prediction is assumed to need at least as long again. Replaying Trace 3 (Fig. 14), interactive response time is zero for every policy from four cores up, one more than the number of CPU-bound jobs. Below that, FCFS and non-preemptive SJF make interactive jobs wait 24–77 ms on average whatever the predictor, because a job that arrives while a CPU-bound job holds the core must wait for it. SRTF with a predictor does not help on this trace at one or two cores: each CPU-bound job runs a single long burst, so no predictor has history to tell it apart; only exact lengths do (0.01 ms). In the VM's two-core captures, where every job has history, SRTF with any predictor cuts mean interactive response from 10.4 ms (FCFS) and 10.2 ms (SJF) to 0.011 ms. On one core, measured EEVDF gave interactive jobs 1.87 ms, against 1.5–1.9 ms for simulated SRTF with predictors, 6.8 ms for Round Robin and 16.4 ms for FCFS. The simulator omits switch and migration costs and per-core queues. Prediction pays only where tasks queue, and then through preemption, which is what slice requests provide.

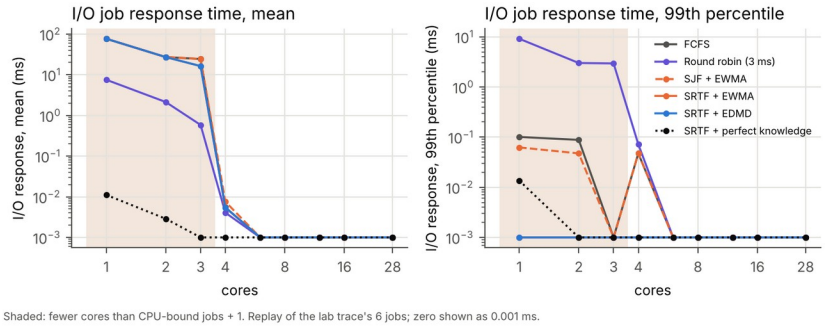


Fig. 14. Simulated interactive response time versus core count, replaying the lab trace's six jobs.

IX. Koopman View of the Whole Machine

The same estimator, applied to a state describing the whole machine, gives a change detector: the one-step residual $r_t = \left(\| z_{t+1} - z_t K \| - \acute{r} \right) / s_r$, in units of the training residual's mean and spread, averaged causally over a short window, raises an alarm when it stays above a threshold for a hold time.

Offline, from traces. The state every 10 ms is the busy fraction and queued tasks of each core plus the CPU share of each job class. In three VM runs where four of six CPU-bound jobs stop at 20 s while every core stays busy, an operator fitted on the first 8 s raised an alarm 77–94 ms after the change in all three, with no false alarm before it or in nine runs without a change (Fig. 15). A persistence residual and a distance-from-normal score on the same inputs detected none of the three; a utilization monitor cannot, because utilization stays at 100%. On the 28-core lab trace, a four-variable machine-wide state detected the first CPU-bound exit 32 ms after it without a false alarm (distance-from-normal: 52 ms).

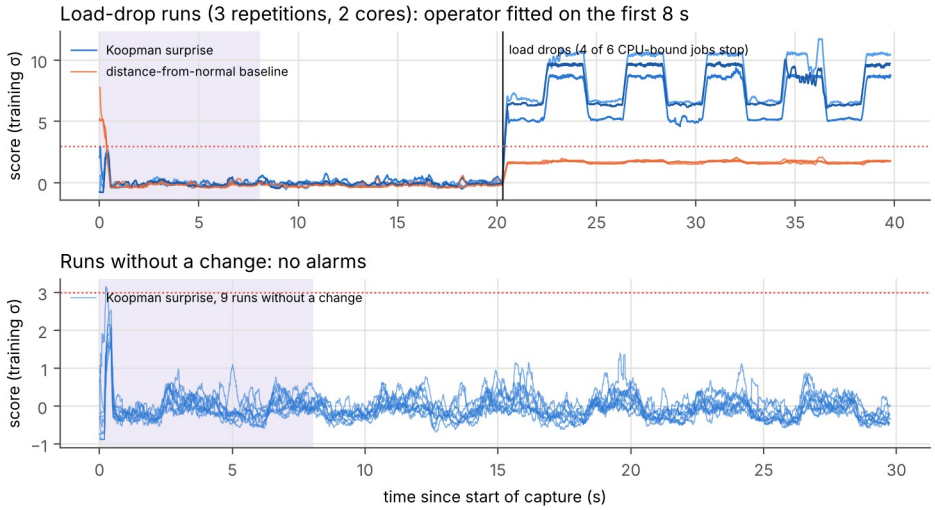


Fig. 15. Trace-based detector on the VM. Top: three runs with a load drop at 20 s. Bottom: nine runs without a change. Shaded: training.

TABLE XI. Live detection at threshold 3: changes detected (of 18), median delay, and runs with a false alarm (of 24), per platform

	ARM VM, 4 cores			Intel VM, 2 cores			Bare metal, 28 CPUs		
Detector	det.	delay	FA	det.	delay	FA	det.	delay	FA
Koopman residual	17	0.12 s	10	12	0.26 s	10	8	0.06 s	6
distance from normal	18	0.11 s	7	16	0.23 s	10	14	0.33 s	6
persistence residual	9	0.08 s	10	7	0.18 s	10	10	0.08 s	8
run-queue monitor	12	0.46 s	0	15	0.20 s	4	14	0.09 s	3
utilization monitor	12	0.46 s	0	6	1.27 s	2	14	0.12 s	5

Live, from counters. A deployable version samples `/proc/stat` every 10 ms, forms a 100 ms state of busy cores, runnable tasks and context switches per second, needs no privileges and no tracing, and fits its operator on the first 8 s. In 24 scripted runs on each VM (at a random time, CPU-bound jobs were added or stopped, or fast I/O jobs started; six runs had no change), it detected 29 of 36 changes with a median delay of 0.18 s, using 0.5–1.1% of one core (Table XI, Fig. 16). A static distance-from-normal score on the same three

variables was as good or better, and across 54 settings of training length, smoothing and threshold the Koopman residual was never strictly better than it. On this low-dimensional state, watching the variables jointly matters more than modelling their dynamics. The same 24 runs on the 28-CPU lab machine (with 14 CPU-bound background jobs) confirm this and show a limit. The Koopman residual detected 8 of 18 changes (median delay 0.06 s; false alarms in 6 of 24 runs; 1.2% of one core): 7 of 9 large changes, in which as many jobs as CPUs were added or stopped or twice as many I/O jobs started, within a median 0.04 s, but only 1 of 9 small changes of a single job. The run-queue and utilization monitors detected 14 of 18 with false alarms in 3 and 5 runs. On a large machine, a one-job change is too small for the residual of a three-variable operator.

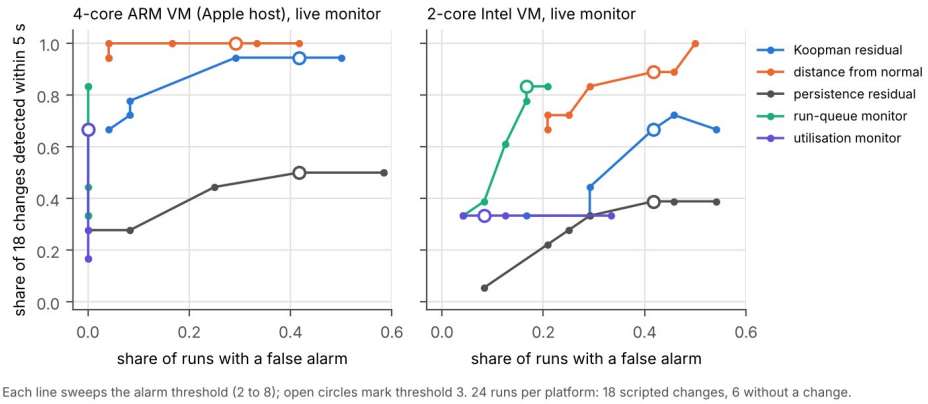


Fig. 16. Live change detection, 24 runs per platform. Each line sweeps the threshold; circles mark threshold 3.

X. Making the Scheduler Visible

The Scheduler Observatory (Fig. 17) replays the validated traces in slow motion in a web browser. Each core is a tile lit in the colour of the task it runs; queued tasks sit under their core with a bar that grows while they wait; sleeping and disk-blocked tasks rest in a pool; migrations are drawn as arcs. A log narrates each scheduling decision with its cause and the wait that preceded it, and a Koopman panel shows, in sync, the eigenvalues of a sliding-window operator on the unit circle and the live residual score. All values come from the reconstructed timelines of Section V. The tool makes the effects in this paper visible: queues forming only when CPU-bound jobs outnumber cores, I/O tasks waiting behind a CPU-bound task that holds its slice, and eigenvalues moving when the load changes.

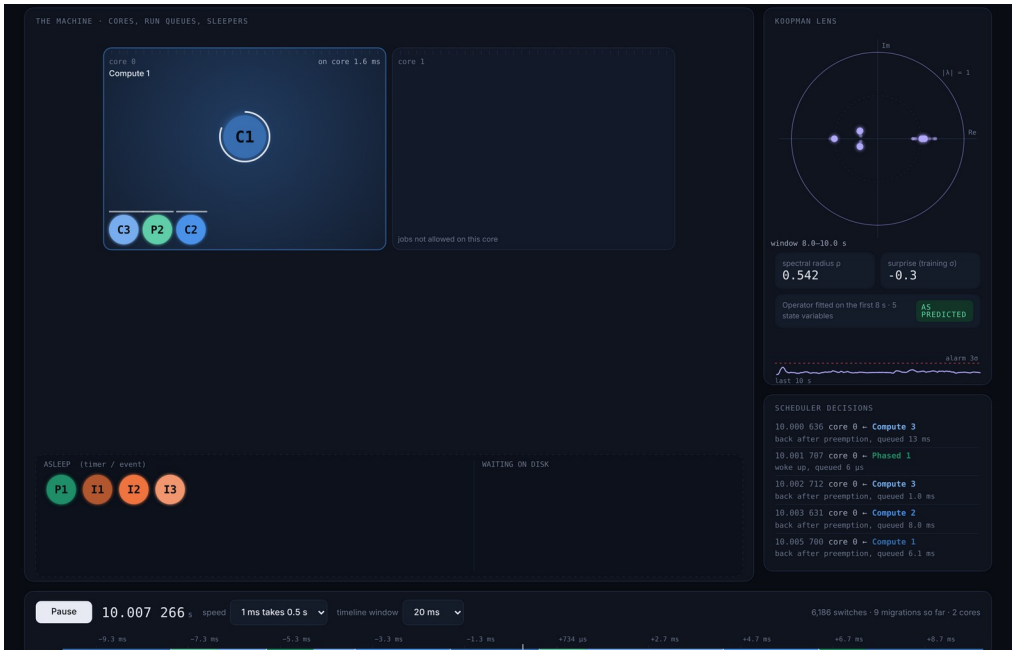


Fig. 17. Scheduler Observatory replaying one core shared by eight jobs: running job, queued jobs with wait bars, sleepers, Koopman eigenvalues and decision log.

XI. Beyond the Operating System

The same estimator, a least-squares operator on a standardized state, has also been applied above the operating system. Two applications are summarized here as case studies of the method.

Network traffic. In earlier work [40], 16.2 million CSE-CIC-IDS2018 flow records [41] were binned into 9,777 windows of 30 s, each summarized by an 11-dimensional state: ports targeted, protocol diversity, flow count, forward and backward volume and their asymmetry, mean and spread of inter-arrival time, and SYN, RST and FIN counts. One 11×11 operator was fitted on 6,871 training transitions that do not cross capture gaps. On the chronologically held-out final day (1,046 transitions) it lowered mean squared forecast error against persistence by 25.9% at 30 s, 29.1% at 60 s and 18.9% at 15 min, and the same construction lowered it by 20.7% on UNSW-NB15 [42]. All eleven eigenvalues are real and inside the unit circle; the largest, 0.995, gives a slowest-mode half-life of 71 min.

Equity markets. EIGENWORLD [43] builds diffusion-map coordinates [44] from seven daily S&P 500 features, fits a Koopman operator on them and tracks the spectral entropy of the embedding. Its kernel bandwidth is drawn from an unseeded random sample, so its unmodified code was rerun under 11 fixed seeds. On the original 2004–2010 data, 10 of 11 seeds reproduce the published result: the minimum spectral entropy falls on 2008-08-27, 19 days before the Lehman Brothers bankruptcy (values 1.960–1.982; published 1.972). Extended to 1992–2026, the lowest entropy in 34 years still falls in July–August 2008 for all 11 seeds (Fig. 18).

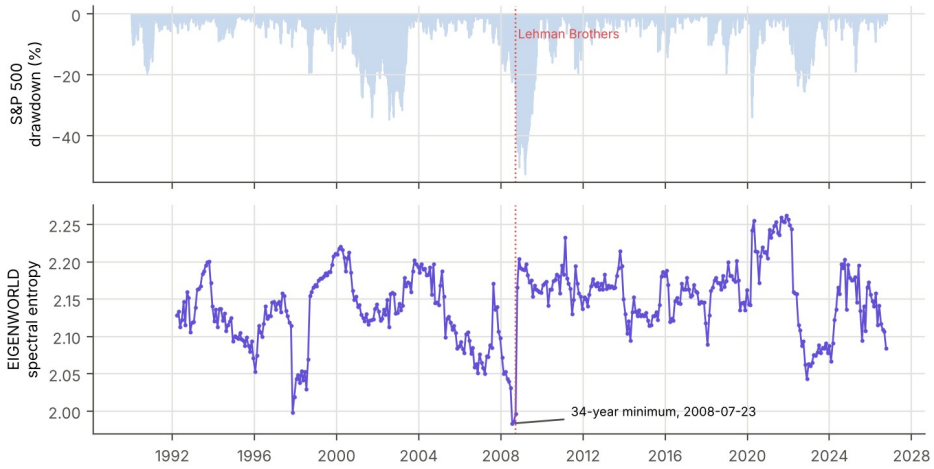


Fig. 18. S&P 500 drawdown and EIGENWORLD's spectral entropy (seed 0), 1992–2026; the 34-year minimum falls in July 2008.

XII. Discussion

TABLE XII. Summary of results

Task	Result	Compared with
Real-program bursts	29% lower error, 32/36 threads	textbook estimator
Contended I/O bursts	14% lower error, 33/33	textbook estimator
Nonlinear observables	no gain	log-AR(p*)
Steering EEVDF, bare metal (1, 8, 16 CPUs)	short requests 57–59% faster, throughput within ±0.8%	stock Linux
Steering EEVDF, VM (1 CPU)	short requests 45% faster, no throughput loss	stock Linux
Koopman vs textbook in loop	better in 48/54 bare-metal and 19/24 VM runs	textbook estimator
Real servers, bare metal	Redis 43%, SQLite points 77%, Node.js light 34% faster; Redis p99 21% slower	stock Linux
Real servers, VM	Redis 31–37%, SQLite points 31% faster; tails unchanged	stock Linux
sched_ext scheduler, bare metal	short requests 44%, long 41% faster; no prediction: 14 times slower	stock EEVDF
Load change, traces	3/3, 77–94 ms	baselines 0/3
Load change, live	29/36 (VMs), 8/18 (28 CPUs)	simple monitors as good or better
I/O slowdown after load drop	gone with `performance` governor	default governor

What the Koopman view contributes. Table XII collects the results. For burst prediction, the Koopman framing earns its place through the log scale, the delay coordinates, the cheap online update and the spectrum, not through nonlinear observables: on scheduler data, Hankel DMD and a log-scale autoregressive model are the same predictor, and richer models add nothing. This is a useful negative result: a practitioner choosing a predictor for a scheduler should pick a log-scale linear model with enough delays and spend the remaining effort elsewhere.

Where the gain comes from. The practical value is in the loop. Predicted bursts, turned into EEVDF slice requests with a margin of about 1.5, made short requests 57–59% faster on contended bare-metal cores with 1, 8 and 16 CPUs (40–45% on the VM), cost nothing measurable in throughput when observed through eBPF, and sped up real servers' short requests by 34–77% on bare metal (about a third on the VM). Used inside a complete sched_ext scheduler, the same predictor turned a policy that was far worse than stock Linux into one that was 41–44% faster. How the forecast becomes a slice matters more than which forecaster is used: the wrong margin erases the gain even for a perfect predictor, and the right one lets an imperfect predictor recover most of it. Within that, a more accurate predictor helps where slices are tight, as the model of Section IV-C predicts; on bare metal, with a default slice twice as long, its advantage held up to $c = 2$. On the polling agent the conformal margin helped the textbook estimator but hurt Koopman, whose wider log-scale intervals pushed slices towards the default; a fixed multiplicative margin worked better for both.

Lessons for practitioners. (1) Validate traces against `schedstat` before trusting them; both hazards found in this study were silent. (2) Predict bursts on a log scale. (3) If a latency-sensitive task's bursts are short, a slice request is a supported, low-risk way to serve it sooner on stock Linux, but the slice must exceed the burst with a margin; a slice equal to the burst can be worse than none. (4) A hand-set minimum slice can beat prediction for a task whose bursts are always short, but hurts tasks whose bursts vary. (5) Watch the medium-sized requests: a confident predictor gives them slices that are too short.

Limits. Heavy requests of real servers did not improve. On bare metal, log-EDMD made medium-sized Redis and SQLite requests 22–32% slower and Redis's 99th percentile 21% slower, and for some programs a hand-set slice was better. For the synthetic workload, the 99th percentile of short requests improved by about a third, while long requests took up to 3% longer with log-EDMD at $c = 1.5$. The gains need contention: on the idle lab machine nothing queued. The margin that worked best here (1.5) depends on the overhead and on the tick and preemption model of the kernel.

Threats to validity. The closed loop with the synthetic workload was confirmed on one bare-metal machine with a hybrid processor, using only its performance cores, and the real-server loop on a second, older bare-metal machine; other processors and kernels may need another margin. The real-server loop covers three servers on one contended core. The sched_ext scheduler is a simple global-queue design tested on one machine with six paired runs; its comparison with stock EEVDF changes the whole scheduler, not only the prediction, which is why the same scheduler without predictions is reported alongside. The closed-loop request workload is synthetic. The bare-metal frequency test supports, but does not prove, the power-management explanation of the slowdown in Trace 3, which came from another machine. The multi-core simulator is idealized; an EEVDF simulator built for this study did not reproduce the measured closed-loop effects and is not used. Network numbers come from the original study, and the market result reproduces one published event.

XIII. Conclusion and Future Work

A Koopman operator on log-scale delay coordinates predicts CPU bursts better than the textbook estimator, by 29% on real programs; it is an autoregressive model in disguise, and nonlinear observables do not improve it. Used to set EEVDF slice requests with a margin, it makes short requests 57–59% faster on the stock Linux scheduler on bare metal with 1, 8 and 16 CPUs at negligible cost, beats the textbook estimator in 48 of 54 paired runs, and makes real servers' short requests 34–77% faster on bare metal, at a cost to some medium-sized requests. A complete sched_ext scheduler driven by the same predictor beat stock EEVDF by 44% on short and 41% on long requests. A simple model explains why the margin is needed and where predictor accuracy matters.

Further directions are a margin that adapts to the spread of each task's recent bursts, which should protect medium-sized requests; per-core queues and load balancing for the sched_ext scheduler; evaluation on more processors; and evaluation on server workloads with service-level objectives.

Data and Code Availability

Capture kit, eBPF observer, controller, sched_ext policy, parsers, validation, simulators, all analyses and data (141 validated captures and counter runs, 69 of them on bare metal, and 540 closed-loop runs, 276 of them on bare metal), the EIGENWORLD reproduction and the visualizer are at https://github.com/SaarthakManocha/Koopman-Operator-Theory-Research; its README lists the script behind each table and figure.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.

[3] The Linux kernel documentation, "CFS scheduler," https://docs.kernel.org/scheduler/sched-design-CFS.html, accessed Oct. 2026.

[4] I. Stoica and H. Abdel-Wahab, "Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation," Old Dominion University, Tech. Rep. TR-95-22, 1995.

[5] J. Corbet, "An EEVDF CPU scheduler for Linux," LWN.net, Mar. 2023, https://lwn.net/Articles/925371/.

[6] J. Corbet, "Completing the EEVDF scheduler," LWN.net, Apr. 2024, https://lwn.net/Articles/969062/.

[7] The Linux kernel documentation, "EEVDF scheduler," https://docs.kernel.org/scheduler/sched-eevdf.html, 2024.

[8] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.

[9] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, no. 1–3, pp. 309–325, 2005.

[10] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.

[11] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, vol. 656, pp. 5–28, 2010.

[12] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.

[13] H. Arbabi and I. Mezić, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM J. Appl. Dyn. Syst.*, vol. 16, no. 4, pp. 2096–2126, 2017.

[14] S. Pan and K. Duraisamy, "On the structure of time-delay embedding in linear models of non-linear dynamical systems," *Chaos*, vol. 30, no. 7, p. 073135, 2020.

[15] R. G. Brown, *Statistical Forecasting for Inventory Control*. New York: McGraw-Hill, 1959.

[16] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959.

[17] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, "On dynamic mode decomposition: Theory and applications," *J. Comput. Dyn.*, vol. 1, no. 2, pp. 391–421, 2014.

[18] M. Korda and I. Mezić, “On convergence of extended dynamic mode decomposition to the Koopman operator,” *J. Nonlinear Sci.*, vol. 28, no. 2, pp. 687–710, 2018.

[19] F. Takens, “Detecting strange attractors in turbulence,” in *Dynamical Systems and Turbulence, Warwick 1980*, ser. Lecture Notes in Mathematics. Springer, 1981, vol. 898, pp. 366–381.

[20] S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, “Chaos as an intermittently forced linear system,” *Nature Commun.*, vol. 8, p. 19, 2017.

[21] S. T. M. Dawson, M. S. Hemati, M. O. Williams, and C. W. Rowley, “Characterizing and correcting for the effect of sensor noise in the dynamic mode decomposition,” *Exp. Fluids*, vol. 57, no. 3, p. 42, 2016.

[22] J. N. Kutz, S. L. Brunton, B. W. Brunton, and J. L. Proctor, *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*. Philadelphia, PA: SIAM, 2016.

[23] J. Mann and J. N. Kutz, “Dynamic mode decomposition for financial trading strategies,” *Quant. Finance*, vol. 16, no. 11, pp. 1643–1655, 2016.

[24] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.

[25] B. Lusch, J. N. Kutz, and S. L. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” *Nature Commun.*, vol. 9, p. 4950, 2018.

[26] B. Gregg, *BPF Performance Tools*. Addison-Wesley, 2019.

[27] The Linux kernel documentation, “Extensible scheduler class (sched_ext),” https://docs.kernel.org/scheduler/sched-ext.html, 2024.

[28] A. Negi and P. K. Kumar, “Applying machine learning techniques to improve Linux process scheduling,” in *Proc. IEEE Region 10 Conf. (TENCON)*, 2005.

[29] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, “Size-based scheduling to improve web performance,” *ACM Trans. Comput. Syst.*, vol. 21, no. 2, pp. 207–233, 2003.

[30] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019, pp. 270–288.

[31] X. Wang, S. Jia, J. Cao, Z. Huang, and M. Song, “Mixture-of-schedulers: An adaptive scheduling agent as a learned router for expert policies,” arXiv:2511.11628, 2025.

[32] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: A decade of wasted cores,” in *Proc. EuroSys*, 2016.

[33] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.

[34] H. Akaike, “A new look at the statistical model identification,” *IEEE Trans. Autom. Control*, vol. 19, no. 6, pp. 716–723, 1974.

[35] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.

[36] A. N. Angelopoulos and S. Bates, “Conformal prediction: A gentle introduction,” *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.

[37] H. R. Künsch, “The jackknife and the bootstrap for general stationary observations,” *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, 1989.

[38] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[39] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[40] S. Manocha, R. R. Lingala, C. S. M. Sadhana, S. Kedia, C. S. M. Sahithi, and P. Vemuri, “Koopman operator theory for network traffic dynamics: An inspectable linear world model for attack forecasting,” 2026, manuscript, Chaitanya Bharathi Institute of Technology.

[41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.

[42] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. Military Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.

[43] S. Manocha, “EIGENWORLD: Causal-topological intelligence for complex systems,” Source code, https://github.com/SaarthakManocha/eigenworld, 2026.

[44] R. R. Coifman and S. Lafon, “Diffusion maps,” *Appl. Comput. Harmon. Anal.*, vol. 21, no. 1, pp. 5–30, 2006.